

Q. Why we required array?

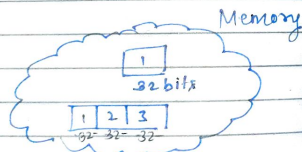
```
int rollNo = 1;
```

```
int roll_no2 = 2;
```

$$\text{inf: roll-no} = 100;$$

This task is so hectic... to solve this hecticness
give us away.

```
int [] rollNumber;
```



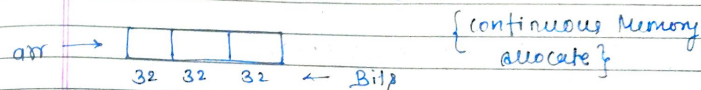
```
int [] roll no = new int [3];
```

↓ ↓ ↓ ↳ size of Array.

datatype
of Array identifier special keyword
allocate memory
in heap

Backend process in compiler.

```
int[] arr = new int[3]
```



⇒ $\left. \begin{array}{l} r1 = 101 \\ r2 = 102 \\ r3 = 103 \end{array} \right\} \text{ int [] roll_no = new int [3];}$

roll_no	101	102	103
	0	1	2

← Index.

rollno[0] = 101; $\rightarrow$

rollno[1] = 102; $\rightarrow$

rollno[2] = 103; $\rightarrow$

```
System.out.println (roll-no[0]); // 101
System.out.println (roll-no[1]); // 102
System.out.println (roll-no[2]); // 103
```

using loop work on arrays.

```
int [] roll-no = new int
```

101	102	103
0	1	2

```

int x = 101;
for (int i = 0; i < 3; i++) {
    arr[i] = x;
    x++;
}

```

print element using loops

```
for (int i = 0; i < 3; i++) {
    System.out.println(rollNo[i]);
}
```

property length of Array: Method.

arr.length → give length of Array.

roll-no

101	102	103
0	1	2

System.out.println(roll-no.length) // 3

remove hard code value

```
for (int i = 0; i < roll-no.length; i++) {
    // 
}
```

Index out of Bound exception.

roll-no =

101	102	103
0	1	2

System.out.println(rollNo[2]); // 103

System.out.println(rollNo[3]);

(Index out of Bound Exception)

Multidimensional Array;

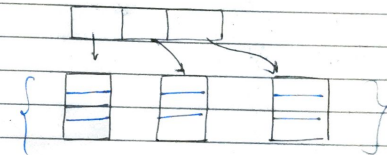
int [] roll = new int [3];



} = 1-D Array
Array of int.

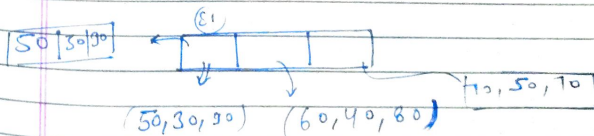
2-D Array

↳ [Array of Arrays]



→ why do we require 2-D Array?

		Hindi	English	CSE
S1 →	students	50	50	90
S2		60	40	80
S3		70	50	70



Definition of Array

`int [][] Marks = new int [3][3];`

No. of rows

No. of columns

	Marks			
Sl	50	30	90	→ r1
S2	60	40	80	→ r2
S3	70	50	70	→ r3

H E CSE (3x3)

C1 C2 C3

(0) (1) (2)
C1 C2 C3

(0) r1	(0,0)	(0,1)	(0,2)
(1) r2	(1,0)	(1,1)	(1,2)
(2) r3	(2,0)	(2,1)	(2,2)

Conceptually how the 2-D Array Store

fill values in 2-D Array.

use nested loop;

Normally

`Marks[0][0] = 50;`
`Marks[0][1] = 30;`
`Marks[0][2] = 90;`
`Marks[1][0] = 60;`
`Marks[1][1] = 40;`
`Marks[1][2] = 80;`
`Marks[2][0] = 70;`
`Marks[2][1] = 50;`
`Marks[2][2] = 70;`

Nested loop.

②

`For (int row = 0; row < 3; row++) {`

`For (int col = 0; col < 3; col++) {`

`System.out.print (Marks [row] [col]);`

`System.out.println ();`

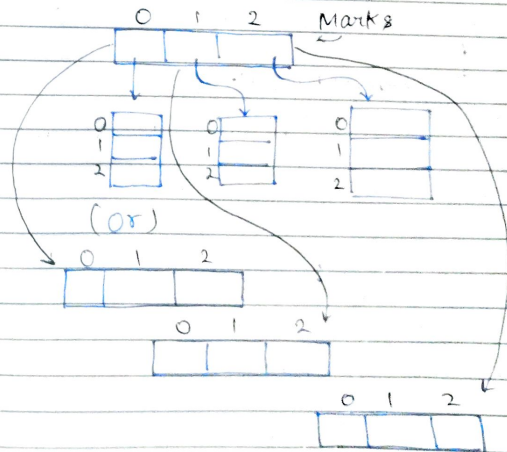
50 30 90

60 40 80

70 50 70

Logical Representation of 2-D Arrays.

`int [][] marks = new int [3][3];`



Marks[0].length
Marks[1].length
Marks[2].length

Marks.length

Remove hard code values of using Logical Rep. of Array.

```
for (int i = 0; i < Marks.length; i++) {  
    for (int j = 0; j < Marks[i].length; j++)  
    {  
        sop(Marks[i][j]);  
    }  
    system.out.println();  
}
```

variable column 2-D array.

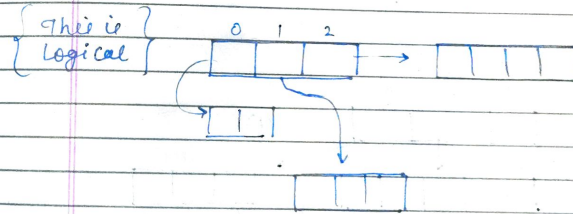
int [][] marks = new int [3] []

↓
H No. of rows
compulsory

↘ optional



marks[0] = new int [2];
marks[1] = new int [3];
marks[2] = new int [4];



How Conceptual representational?

		0	1	2	3
		c ₁	c ₂	c ₃	c ₄
0	r ₁	(0,0)	(0,1)	↓	
1	r ₂	(1,0)	(1,1)	(1,2)	↓
2	r ₃	(2,0)	(2,1)	(2,2)	(2,3)

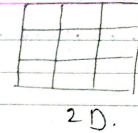
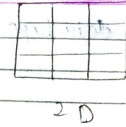
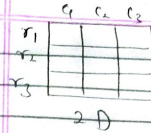
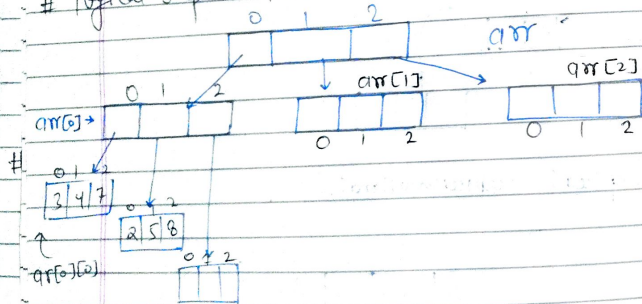
alternate representation.

`int Marks[3][3] = new int[3][3];`

3-D Array;

`int [][][3] arr = new int [3][3][3];`

logical representation



4D, 5D and:

Multi Dimensional Arrays

Other way to represent Array Declaration.

① 1-D

`int [3] roll = new int [3];`

`roll[0] = 4;`
`roll[1] = 5;`

`int [3] roll = { 4, 5, 6 };`

② 2-D

`int [3][3] Marks = {`

`{ 4, 5, 6 }`
`{ 7, 8, 9 }`
`{ 10, 12, 30 }`
`};`

String in java;
↳ Non-primitive data type.

```
String name = "Manish";  
String name2 = "Saini";  
System.out.println ( name + name2 );
```

result.

Manish_Saini.